

ESDT Scan2BIM Inshelter Pipeline Architecture

The Inshelter Scan2BIM pipeline converts a single E57 scan of a telecom shelter interior into an IFC model. It is optimized for indoor, enclosed spaces and differs from the full-site pipeline by using wall-aware refinement, two-pass segmentation, and a dedicated .NET IFC creation service. This page documents the architecture, flow, interfaces, processing stages, and operational considerations for the single-scan Inshelter pipeline.

- Primary input:** single-face E57 point cloud scan
- Primary output:** IFC BIM model of shelter interior components
- Main components detected:** walls, doors, racks, cabinets, batteries, rectifiers, floors, and ceilings

- Why this pipeline matters:** shelter interiors are dense, bounded, and geometry-sensitive. The two-pass design improves precision by first understanding the overall space and then refining inference inside wall-bounded areas.

- [Architecture overview](#)
- [Flowchart](#)
- [End-to-end pipeline flow](#)
- [Core design details](#)
 - [Why the two-pass segmentation matters](#)
 - [Communication patterns](#)
 - [Representative payloads](#)
- [Low-level implementation notes](#)
- [Operations, resilience, and observability](#)
 - [Error handling and retry model](#)
 - [Infrastructure highlights](#)
 - [Monitoring priorities](#)

Architecture overview

The solution is built as a distributed microservice workflow coordinated by a root orchestration service. Processing begins when a user uploads an E57 file through the ESDT Portal. DataRepo stores the file and triggers a webhook to the root microservice, which starts a Temporal workflow, generates a SAS download URL, dispatches segmentation, tracks post-processing and IFC creation, and persists workflow state in MongoDB.

The system combines asynchronous queue-based processing with synchronous workflow control. Azure Service Bus is used for durable message passing, Azure Blob Storage holds large

artifacts such as E57, segmented PLY, and IFC files, and HTTP activity signals are used to notify workflow completion milestones.

Service	Technology	Primary role	Key outputs
Root Orchestrator	TypeScript, NestJS, Temporal client	Accepts webhook events, starts workflows, routes messages, tracks status	Workflow state, queue messages, completion handling
Segmentation	Python, GPU processing	Runs two-pass ML segmentation, wall extraction, image extraction	Segmented PLY, wall JSON, pass-2 output
Post-Processing	Python	Extracts geometry for BIM-relevant components	Geometry JSON
IFC Creation	C# .NET, xBIM	Maps geometry JSON into IFC entities and creates BIM file	IFC file

Flowchart

The following diagram is derived from the provided draw.io XML and represents the end-to-end pipeline architecture and message flow.

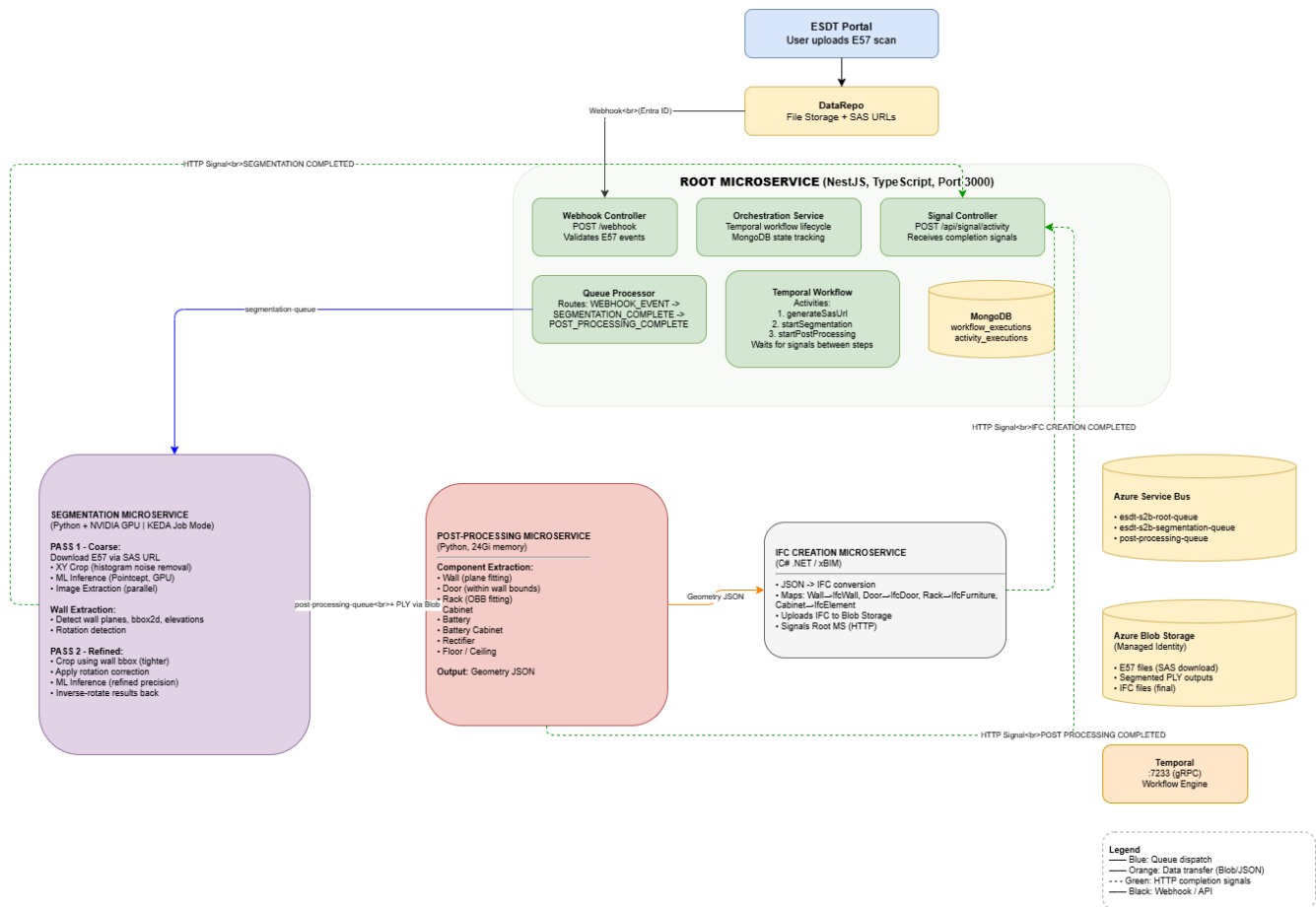


Diagram interpretation: blue arrows show queue dispatch, orange arrows show artifact or geometry transfer, green dashed arrows show HTTP completion signals, and black arrows show webhook or API entry points.

End-to-end pipeline flow

- Upload and event creation:** A user uploads an E57 shelter scan through the ESDT Portal. DataRepo stores the file and emits an authenticated webhook event.
- Workflow initiation:** The Root Microservice validates key fields such as file type, file ID, customer ID, and site ID. It generates a workflow identifier, stores an initial record in MongoDB, and publishes a `WEBHOOK_EVENT` message to the root queue.
- Temporal orchestration:** The queue processor consumes the event and starts the `scan2bimWorkflow` in Temporal. Workflow status moves to running.
- SAS URL generation:** A synchronous workflow activity requests a download SAS URL from DataRepo so downstream services can securely retrieve the E57 file.

5. **Segmentation dispatch:** The orchestrator publishes a segmentation start message to the segmentation queue. The workflow then waits for completion signals.
6. **Two-pass segmentation:** The segmentation service downloads the E57 file, performs coarse inference, extracts walls, computes wall bounds and optional rotation, and then runs a refined second-pass inference within wall-aware boundaries.
7. **Artifact publication:** Segmentation uploads outputs to Blob Storage, sends a completion event to the root queue, and pushes a post-processing start message to the post-processing queue.
8. **Geometry extraction:** Post-processing downloads segmented artifacts and derives BIM-ready geometry for walls, doors, racks, cabinets, batteries, rectifiers, floors, and ceilings.
9. **IFC generation:** The IFC service converts geometry JSON into IFC entities using xBIM and uploads the result to Blob Storage.
10. **Completion:** Root receives activity signals, updates MongoDB state, and completes the Temporal workflow. The IFC file is then available for downstream consumption.

Core design details

Why the two-pass segmentation matters

Indoor shelter scans often contain scanner noise, occlusions, and orientation variance. A single-pass inference can detect major classes, but it may not provide the precision needed for BIM-grade geometry extraction. The pipeline therefore uses two passes:

- **Pass 1:** operates on the broader scan after XY cropping to identify coarse semantic classes and establish a structural understanding of the room.
- **Wall extraction:** uses pass-1 output to detect wall planes, compute 2D and 3D bounds, determine elevation ranges, and identify whether the scan should be rotated into an axis-aligned reference frame.
- **Pass 2:** focuses on a tighter, wall-bounded region and produces higher-quality segmentation for downstream geometry extraction.

✓ **Practical benefit:** this design improves component localization, reduces the influence of irrelevant points, and produces cleaner input for geometry extraction and IFC mapping.

Communication patterns

From	To	Method	Purpose
------	----	--------	---------

DataRepo	Root Microservice	HTTP webhook	Notifies file upload event
Root Microservice	Segmentation	Azure Service Bus queue	Starts segmentation job
Segmentation	Root Microservice	Azure Service Bus queue	Reports segmentation completion
Segmentation	Post-Processing	Azure Service Bus queue plus Blob artifact reference	Triggers geometry extraction
Post-Processing	Root Microservice	HTTP signal	Reports post-processing completion
IFC Creation	Root Microservice	HTTP signal	Reports IFC completion
Root Microservice	Temporal	gRPC	Starts workflow and forwards signals
All services	Blob Storage / MongoDB	SDK or driver	Persist artifacts and status

Representative payloads

The workflow depends on structured queue and signal payloads. The most important message types are:

- `WEBHOOK_EVENT` to initiate workflow execution
- `SEGMENTATION_START` to launch GPU processing
- `SEGMENTATION_COMPLETE` to publish segmented outputs and wall metadata
- `POST_PROCESSING_AND_UPLOAD_START` to trigger component extraction
- `POST_PROCESSING_AND_UPLOAD_COMPLETE` to publish geometry and IFC output references
- HTTP activity signal payloads for Temporal state progression

▼ Example activity signal payload

```
1 {
2   "workflowId": "scan2bim-abc123",
3   "activityName": "SEGMENTATION",
```

```
4  "status": "COMPLETED",
5  "output": {
6    "segmentedFileUrl": "https://...",
7    "pointCount": 1500000
8  }
9 }
```

Low-level implementation notes

The root service centers on four key modules: webhook handling, orchestration, queue processing, and signal forwarding. The Temporal workflow performs deterministic coordination and waits for external completion signals between major stages. MongoDB stores workflow and activity execution records so operators can query progress and diagnose failures.

The segmentation service is designed to run in job mode with GPU acceleration. Its responsibilities include E57 retrieval, cropping, inference, wall extraction, refined inference, output upload, and downstream dispatch. Post-processing then applies component-specific extractors, each tailored to the geometry and constraints of a different BIM object class. Finally, the IFC service maps extracted geometry into IFC entity types such as `IfcWall`, `IfcDoor`, and `IfcFurniture`.

Operations, resilience, and observability

Error handling and retry model

Failure point	Expected behavior	Outcome
Segmentation failure, including GPU memory issues	Failure signal is sent or processing terminates after retry policy	Workflow marked failed in MongoDB
Post-processing failure	Activity fails after retry policy	Workflow marked failed
IFC creation failure	Failure signal or unrecoverable service error	Workflow marked failed
No walls detected	Pass 2 is skipped and pipeline continues with pass-1 output	Degraded but continued processing

Poison queue message	Service Bus retries are exhausted	Message routed to dead-letter queue
----------------------	-----------------------------------	-------------------------------------

⚠ **Important operational nuance:** no-wall scenarios are handled differently from hard failures. They trigger a fallback path rather than stopping the workflow outright.

Infrastructure highlights

- **Root service:** moderate CPU and memory, no GPU required
- **Segmentation:** highest resource demand, including NVIDIA GPU and very high memory ceiling
- **Post-processing:** memory-heavy due to geometry extraction on dense point cloud outputs
- **IFC creation:** comparatively lightweight but critical for final deliverable generation

Monitoring priorities

- **Workflow duration:** monitor webhook-to-completion time with a target under 90 minutes
- **Per-stage timing:** track pass-1 inference, pass-2 inference, post-processing, and IFC creation separately
- **Correlation:** include `workflowId` in all logs, queue messages, and activity records
- **State inspection:** use MongoDB collections for execution history and activity-level debugging

Useful operational checks

Workflow record query: inspect `workflow_executions` by `workflowId` to confirm current status and timestamps.

Activity record query: inspect `activity_executions` to identify where execution is waiting, failed, or completed.

Queue health: verify active, deferred, and dead-letter counts for root, segmentation, and post-processing queues.

Summary: the Inshelter pipeline is a workflow-driven, multi-service architecture that combines GPU inference, wall-aware refinement, geometry extraction, and IFC generation into a resilient BIM production path for single-scan shelter interiors.